

Roguia : Une Approche par Apprentissage par Renforcement Hybride dans un Roguelike

Documentation Technique et Formalisation Mathématique

Auteurs et Équipe Technique :

Sabri Messaoudi · Chrisphen Ringuet · Raykesh Ravirooban
Coumba Mbaye · Maxence Klein

Sous la supervision de :

Armen Petrossian & Ronan Costaouec

ESEO - Option DAIA

16 janvier 2026

Résumé

Ce document technique détaille l'implémentation d'agents autonomes basés sur l'apprentissage par renforcement (RL) au sein du projet *Roguia*. Nous formulons le problème de navigation en environnement stochastique comme un Processus de Décision Markovien (MDP) à états discrets bornés. Nous proposons une architecture hybride novatrice combinant un apprentissage en ligne (*Online Learning*) via l'algorithme Q-Learning tabulaire, et une consolidation hors-ligne (*Batch RL*) permettant l'agrégation d'expériences multi-agents. Les résultats expérimentaux démontrent une convergence de la politique optimale sur un espace d'état réduit de 169 dimensions, validée par l'analyse des cartes de chaleur (heatmaps) de la fonction de valeur Q .

Mots-clés : Apprentissage par Renforcement, Q-Learning, MDP, Batch RL, Roguelike, Reward Shaping, PBRs.

Table des matières

1	Introduction	3
1.1	Objectifs du Projet	3
2	Modélisation du Problème (MDP)	3
2.1	Algorithme de Génération Procédurale	4
2.2	Dynamique de l'Environnement et Stochasticité	4
2.3	Espace d'État $\mathcal{S}$	4
2.4	Espace d'Action $\mathcal{A}$	5
2.5	Ingénierie de la Récompense (Reward Shaping)	5
2.5.1	Définition du Potentiel Φ	5
2.5.2	Récompenses Environnementales R_{env}	5
2.6	Paramètres de Simulation et Progression	5
3	Algorithme d'Apprentissage	6
3.1	Q-Learning Tabulaire	6
3.2	Hyperparamètres	6
3.3	Stratégie d'Exploration ϵ -Greedy	7
3.4	Initialisation et Pré-entraînement (Cold Start)	7
3.5	Synthèse : Boucle de Décision Temps-Réel	7
4	Discussion Théorique	8
4.1	Propriété d'Invariance par Translation	8
4.2	Justification de l'Architecture Batch : L'apprentissage Off-Policy	8
5	Architecture Distribuée et Batch RL	9
5.1	Pipeline ETL et Base de Données	9
5.2	Algorithme Batch Q-Learning	9
5.3	Mécanisme d'Experience Replay	9
5.4	Déploiement et Injection de Politique	10
5.5	Analyse de Complexité	10
5.6	Optimisation : Apprentissage Parallèle Synchron	10
5.7	Dynamique Multi-Agents et Évitement Social	10
6	Analyse Expérimentale	11
6.1	Comparaison par rapport à la Baseline	11
6.2	Métriques de Performance (KPI)	11
6.3	Convergence de la Fonction de Valeur	11
6.4	Impact du Reward Shaping	12
7	Conclusion	13
7.1	Perspectives d'Évolution	13
A	Annexes	15
A.1	Architecture Logicielle	15
A.2	Procédure de Déploiement Utilisateur	15
A.3	Chaîne d'Industrialisation (Packaging)	15

1 Introduction

Ce document présente la formalisation mathématique et l'implémentation technique des agents autonomes au sein du projet *Roguia*. L'objectif est de développer une intelligence artificielle capable de contrôler des entités ennemies ("monstres") dans un environnement stochastique généré procéduralement.

L'approche retenue repose sur l'Apprentissage par Renforcement (RL), et plus spécifiquement sur l'algorithme Q-Learning [1]. Le système se distingue par une architecture hybride combinant :

- Un **Apprentissage en ligne (Online)** : Local à chaque instance de jeu, permettant une adaptation rapide via une Q-Table partagée en mémoire (`shared_q_data`).
- Un **Apprentissage hors ligne (Batch)** : Globalisé, permettant l'émergence d'un "Super Monstre" via l'agrégation de données massives stockées sur MongoDB.

1.1 Objectifs du Projet

Le projet *Roguia* répond à une triple ambition technique, ludique et pédagogique :

- **Objectif Technique (IA & Data)** : Concevoir une architecture d'Intelligence Artificielle hybride capable d'apprendre en continu (*Lifelong Learning*). Le défi est de coupler un apprentissage temps-réel (adaptation rapide) avec une consolidation hors-ligne sur des données massives (robustesse).
- **Objectif Ludique (Game Design)** : Proposer une expérience de jeu "Roguelike" où la difficulté n'est pas scriptée, mais émergente. L'IA doit s'adapter organiquement au niveau du joueur pour maintenir un état de *Flow* (ni trop facile, ni trop dur).
- **Objectif Ingénierie** : Déployer un pipeline de données complet (End-to-End) intégrant la collecte de logs hétérogènes (ETL), le stockage NoSQL (MongoDB) et le déploiement de modèles sérialisés en production.

Environnement Technique : Le projet s'appuie sur un écosystème Python modulaire. Les bibliothèques principales utilisées sont :

- **Moteur de Jeu** : `pygame` pour la boucle de rendu, la gestion des sprites et les événements utilisateurs.
- **Intelligence Artificielle & Data** :
 - `numpy` et `pandas` pour la manipulation vectorielle et le traitement des logs.
 - `matplotlib` et `seaborn` pour la génération des graphiques d'analyse (Heatmaps, KPIs).
- **Persistance & Backend** :
 - `pymongo` : Driver pour la base NoSQL MongoDB (Big Data/IA).
 - `psycopg2` : Adaptateur pour la base relationnelle PostgreSQL (Progression Joueur).
- **Sécurité & Configuration** : `bcrypt` pour le hachage des mots de passe et `python-dotenv` pour la gestion des variables d'environnement.

2 Modélisation du Problème (MDP)

Le problème de navigation et d'attaque des monstres est modélisé comme un Processus de Décision Markovien (MDP) défini par le tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$.

La dynamique de transition $\mathcal{P}(\mathbf{s}'|\mathbf{s}, a)$ étant déterminée par la génération procédurale du donjon et donc inconnue *a priori*, une approche *Model-Free* est nécessaire.

2.1 Algorithme de Génération Procédurale

La topologie du donjon $\mathcal{M}$ est générée via un processus stochastique séquentiel garantissant la solvabilité (existence d'un chemin $\text{Start} \rightarrow \text{End}$).

Placement des Salles (Rejection Sampling) : Soit $\mathcal{R} = \{r_1, \dots, r_N\}$ l'ensemble des salles. Chaque salle r_i est définie par un tuple (x, y, w, h) échantillonné selon une distribution uniforme sur l'espace grille $\mathcal{G} \subset \mathbb{Z}^2$:

$$(x, y) \sim \mathcal{U}(0, W - w), \quad (w, h) \sim \mathcal{U}(w_{\min}, h_{\max}) \quad (1)$$

Une salle candidate r_{new} est acceptée si et seulement si elle ne chevauche aucune salle existante avec une marge de sécurité δ :

$$\mathcal{R}_{k+1} = \mathcal{R}_k \cup \{r_{\text{new}}\} \iff \forall r_j \in \mathcal{R}_k, \quad \text{IoU}(r_{\text{new}} \oplus \delta, r_j) = 0 \quad (2)$$

Connectivité (Graphe Linéaire) : Pour garantir la connexité, les salles sont reliées selon une structure de graphe linéaire (Path Graph) $G = (V, E)$ où $V = \mathcal{R}$ et $E = \{(r_i, r_{i+1}) \mid 1 \leq i < N\}$. Des couloirs $C_{i,i+1}$ sont creusés par un algorithme de "Dog-leg" (chemin en L) ou via un BFS contraint, connectant le centroïde de r_i à celui de r_{i+1} .

Cette méthode assure qu'il existe toujours au moins une trajectoire valide τ reliant le point de départ $S \in r_1$ au point d'arrivée $E \in r_N$, condition nécessaire pour la convergence de l'apprentissage par renforcement.

2.2 Dynamique de l'Environnement et Stochasticité

L'environnement $\mathcal{E}$ dans lequel évoluent les agents n'est pas statique. À chaque nouvel épisode k , une nouvelle instance de carte $\mathcal{M}_k$ est générée par un algorithme procédural (voir `map.py`).

L'espace des positions valides $\mathcal{W} \subset \mathbb{Z}^2$ est défini par l'union des ensembles de salles $\mathcal{R}$ et de couloirs $\mathcal{C}$:

$$\mathcal{W}_k = \left(\bigcup_{i=1}^{N_{\text{rooms}}} \mathcal{R}_i \right) \cup \mathcal{C}_{\text{connect}} \quad (3)$$

La fonction de transition de probabilité $\mathcal{P}(\mathbf{s}'|\mathbf{s}, a)$ du MDP dépend donc explicitement de la topologie locale générée $\mathcal{M}_k$. Cette variabilité structurelle impose à l'agent d'apprendre une politique π^* généralisable aux caractéristiques locales (coins, couloirs, espaces ouverts) plutôt que de mémoriser une carte spécifique (overfitting), ce qui justifie l'utilisation d'un espace d'état relatif egocentrique.

2.3 Espace d'État $\mathcal{S}$

Contrairement à une approche par vision par ordinateur (pixels) qui induirait une explosion combinatoire, nous optons pour une représentation centrée sur l'agent (egocentrique). Soit $\mathbf{p}_m = (x_m, y_m)$ la position du monstre et $\mathbf{p}_p = (x_p, y_p)$ la position du joueur dans la grille discrète $\mathbb{Z}^2$.

L'état $\mathbf{s}_t \in \mathcal{S}$ à l'instant t est défini par le vecteur de différence relative, projeté sur un espace borné :

$$\mathbf{s}_t = \begin{bmatrix} \text{clip}(x_p - x_m, -\kappa, \kappa) \\ \text{clip}(y_p - y_m, -\kappa, \kappa) \end{bmatrix} \quad (4)$$

Où la fonction de saturation clip est définie par :

$$\text{clip}(z, a, b) = \min(\max(z, a), b) \quad (5)$$

Dans notre implémentation (`rl.py`), $\kappa = 6$, ce qui limite la perception locale de l'agent à une fenêtre de $(2\kappa + 1) \times (2\kappa + 1)$ cases. L'espace d'état résultant a une cardinalité de $|\mathcal{S}| = 13^2 = 169$ états.

2.4 Espace d'Action $\mathcal{A}$

L'agent dispose d'un espace d'action discret comprenant les quatre directions cardinales :

$$\mathcal{A} = \{\text{HAUT}, \text{BAS}, \text{GAUCHE}, \text{DROITE}\} \cong \{0, 1, 2, 3\} \quad (6)$$

Chaque action $a \in \mathcal{A}$ induit un déplacement potentiel $\Delta \mathbf{p}$.

2.5 Ingénierie de la Récompense (Reward Shaping)

La fonction de récompense est critique pour la convergence. Nous utilisons le *Potential-Based Reward Shaping* (PBRS) pour densifier le signal sans altérer la politique optimale, conformément au théorème de Ng et al. [3].

La récompense immédiate r_t est la somme d'une composante environnementale et d'un potentiel de mise en forme $\mathcal{F}$:

$$r_t(\mathbf{s}, a, \mathbf{s}') = R_{\text{env}}(\mathbf{s}, a, \mathbf{s}') + \underbrace{\gamma \Phi(\mathbf{s}') - \Phi(\mathbf{s})}_{\text{Terme de Shaping } \mathcal{F}} \quad (7)$$

2.5.1 Définition du Potentiel Φ

Nous définissons le potentiel $\Phi(\mathbf{s})$ comme l'inverse de la distance de Manhattan L_1 vers le joueur. Dans l'implémentation, cela se traduit par une récompense différentielle :

$$\mathcal{F}(\mathbf{s}_t, \mathbf{s}_{t+1}) \approx \beta \cdot [d(\mathbf{s}_t) - d(\mathbf{s}_{t+1})] \quad (8)$$

avec $\beta = 0.6$ et $d(\mathbf{s}) = |x_p - x_m| + |y_p - y_m|$.

2.5.2 Récompenses Environnementales R_{env}

- **Coût d'existence** : -1.0 par pas (incitation à la rapidité).
- **Pénalité d'obstacle** : -5.0 (collision murale).
- **Pénalité sociale** : -2.0 (collision entre monstres).
- **Récompense terminale** : $+20.0$ (capture du joueur).

2.6 Paramètres de Simulation et Progression

Au-delà de la topologie, la complexité de la tâche est régie par des variables d'état internes aux agents et une courbe de difficulté globale.

Hétérogénéité des Agents (Normal vs Super) : La population d’agents $\mathcal{P}$ est composée de deux classes distinctes, influençant la dynamique des épisodes :

- **Monstres Standards** (m_{std}) : Initialisés avec des attributs stochastiques faibles ($PV \sim \mathcal{U}(50, 70)$). Ils servent principalement à générer du bruit et de l’obstacle social.
- **Super Monstres** (m_{sup}) : Agents d’élite ($PV > 100$, Dégâts $\times 1.5$) instanciés uniquement lorsque la profondeur du donjon $d > 2$. Ils agissent comme des points de singularité dans l’apprentissage, nécessitant une politique plus robuste de la part du joueur, et donc indirectement, une meilleure adaptation de l’IA pour survivre (ou tuer).

Fonction de Difficulté Croissante : L’environnement est non-stationnaire en termes de densité. La difficulté Δ est une fonction monotone de la profondeur du niveau d :

$$N_{agents}(d) = \lfloor 2 + \frac{d}{2} \rfloor, \quad \mathbb{P}(spawn_{super}) = \min(1.0, \max(0, (d - 2) \cdot 0.1)) \quad (9)$$

Cette densification progressive de l’espace $\mathcal{W}$ augmente mécaniquement la probabilité de collisions sociales et réduit l’espace libre, forçant l’algorithme RL à optimiser les trajectoires fines au fil de la progression.

Système d’Inventaire et Portée : Bien que l’état $\mathbf{s}_t$ soit positionnel, l’interaction est modulée par l’équipement. Chaque agent possède une portée d’interaction δ_{weapon} (ex : 1.0 pour une épée, 5.0 pour un arc). Actuellement, le MDP est optimisé pour une politique de contact ($\delta_{weapon} \approx 1$), mais l’architecture permet d’intégrer cette variable dans l’état futur pour apprendre des comportements de *Kiting* (attaque à distance et repli).

3 Algorithme d’Apprentissage

3.1 Q-Learning Tabulaire

L’agent apprend la fonction de valeur d’action optimale $Q^*(\mathbf{s}, a)$ via l’équation de Bellman d’optimalité. La règle de mise à jour appliquée à chaque pas de temps t est :

$$\delta_t = r_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q(\mathbf{s}_{t+1}, a') - Q(\mathbf{s}_t, a_t) \quad (10)$$

$$Q(\mathbf{s}_t, a_t) \leftarrow Q(\mathbf{s}_t, a_t) + \alpha \cdot \delta_t \quad (11)$$

Où δ_t représente l’erreur de différence temporelle (TD-Error).

3.2 Hyperparamètres

Les hyperparamètres retenus (cf. `engine/rl.py`) sont :

Paramètre	Symbole	Valeur
Learning Rate	α	0.2
Discount Factor	γ	0.95
Exploration initiale	ϵ_{start}	0.5
Exploration minimale	ϵ_{min}	0.05
Decay Factor	λ_ϵ	0.992
Horizon de vue	κ	6

TAB. 1 – Hyperparamètres de l’agent Q-Learning

3.3 Stratégie d’Exploration ϵ -Greedy

La politique suivie $\pi(\mathbf{s})$ est ϵ -greedy, assurant un compromis exploration/exploitation :

$$a_t = \begin{cases} \text{uniforme}(\mathcal{A}) & \text{si } \xi < \epsilon_t \\ \arg \max_{a \in \mathcal{A}} Q(\mathbf{s}_t, a) & \text{sinon} \end{cases} \quad (12)$$

avec $\xi \sim \mathcal{U}[0, 1]$ et une décroissance exponentielle $\epsilon_{t+1} = \max(\epsilon_{min}, \epsilon_t \cdot \lambda_\epsilon)$.

Convergence Théorique (Condition GLIE) : La stratégie de décroissance exponentielle de ϵ vise à satisfaire approximativement la propriété **GLIE** (*Greedy in the Limit with Infinite Exploration*). Cette condition est nécessaire pour garantir la convergence asymptotique de l’algorithme Q-Learning vers la fonction de valeur optimale Q^* :

$$\lim_{t \rightarrow \infty} \epsilon_t = 0 \quad \text{et} \quad \sum_{t=1}^{\infty} \epsilon_t = \infty \quad (13)$$

Dans notre implémentation, nous maintenons toutefois un $\epsilon_{min} = 0.05$ résiduel pour conserver une plasticité face à la non-stationnarité de l’environnement (génération procédurale).

3.4 Initialisation et Pré-entraînement (Cold Start)

Une initialisation uniforme $Q_0(\mathbf{s}, a) = 0$ entraînerait une phase d’exploration purement stochastique (marche aléatoire) au début de l’interaction avec le joueur, nuisant à l’expérience de jeu.

Pour pallier ce problème de démarrage à froid (*Cold Start*), nous procédons à une phase de pré-entraînement simulé (*Bots vs Bots*). Cette méthode permet d’injecter une connaissance *a priori* dans la matrice Q avant le déploiement :

$$Q_{deploy} \leftarrow \text{Train}(Q_0, \text{SimulatedEpisodes}) \quad (14)$$

L’agent débute ainsi la partie avec une politique π_{init} déjà capable de se diriger grossièrement vers la cible.

3.5 Synthèse : Boucle de Décision Temps-Réel

L’algorithme exécuté à chaque tick de jeu par chaque entité est formalisé ci-dessous. Il intègre la gestion de la mémoire partagée et la politique d’exploration.

Algorithm 1 Cycle de Vie d'un Agent (Online)

```
1: Initialisation :  $\mathbf{s} \leftarrow$  Perception de l'environnement
2: while Agent est vivant do
3:   1. Décision ( $\epsilon$ -Greedy) :
4:   if  $\text{rand}() < \epsilon$  then
5:      $a \leftarrow$  Choix Aléatoire
6:   else
7:      $a \leftarrow \arg \max_{a'} Q_{\text{shared}}(\mathbf{s}, a')$ 
8:   end if
9:   2. Action : Exécuter  $a$ , observer  $r$  et nouvel état  $\mathbf{s}'$ 
10:  3. Apprentissage Local (TD-Update) :
11:   $\delta \leftarrow r + \gamma \max_{a'} Q_{\text{shared}}(\mathbf{s}', a') - Q_{\text{shared}}(\mathbf{s}, a)$ 
12:   $Q_{\text{shared}}(\mathbf{s}, a) \leftarrow Q_{\text{shared}}(\mathbf{s}, a) + \alpha \cdot \delta$ 
13:  4. Collecte (Log) :
14:   $\text{Buffer.push}(\mathbf{s}, a, r, \mathbf{s}', \text{done})$ 
15:   $\mathbf{s} \leftarrow \mathbf{s}'$ 
16: end while
```

4 Discussion Théorique

4.1 Propriété d'Invariance par Translation

Le choix d'un espace d'état relatif $\mathbf{s} = \mathbf{p}_p - \mathbf{p}_m$ (au lieu de positions absolues) est fondamental. Il confère à la fonction de valeur une propriété d'invariance spatiale :

$$\forall \mathbf{v} \in \mathbb{Z}^2, \quad Q^*(\mathbf{p}_m + \mathbf{v}, \mathbf{p}_p + \mathbf{v}) = Q^*(\mathbf{p}_m, \mathbf{p}_p) \quad (15)$$

Cette factorisation exploite les symétries de l'environnement euclidien (grille), permettant une *généralisation structurelle* : une stratégie d'encerclement apprise dans le quadrant Nord-Ouest est immédiatement applicable dans le quadrant Sud-Est, réduisant drastiquement la complexité d'échantillonnage nécessaire à la convergence.

4.2 Justification de l'Architecture Batch : L'apprentissage Off-Policy

L'architecture "Super Monster" repose sur la réutilisation de trajectoires historiques $\mathcal{D}$ stockées en base de données. La validité mathématique de cette approche repose sur la nature **Off-Policy** de l'algorithme Q-Learning.

Contrairement aux méthodes *On-Policy* (ex : SARSA) qui nécessitent que les données soient générées par la politique en cours d'évaluation, Q-Learning approxime la fonction optimale Q^* indépendamment de la politique comportementale μ ayant généré les données, sous la seule condition que μ explore suffisamment l'espace d'états :

$$\lim_{t \rightarrow \infty} Q(s, a) = Q^*(s, a) \quad \text{avec échantillonnage } a \sim \mu(\cdot | s) \quad (16)$$

Cela nous permet d'agréger des millions de transitions provenant d'agents aux niveaux de compétence hétérogènes (débutants à experts) pour raffiner une politique globale robuste.

5 Architecture Distribuée et Batch RL

Le projet *Roguia* implémente une boucle d'apprentissage fermée entre les clients de jeu et un serveur centralisé.

5.1 Pipeline ETL et Base de Données

Les transitions de jeu sont capturées et stockées selon le schéma suivant :

1. **Client** : Chaque monstre instancie un `QLearningAgent`. Si l'option `shared_q_data` est active (voir `monster.py`), tous les monstres d'une même partie partagent la même référence mémoire pour la matrice Q , accélérant l'apprentissage local.
2. **Persistance** : Les transitions $(\mathbf{s}, a, r, \mathbf{s}', \text{done})$ sont exportées vers une base NoSQL MongoDB.
3. **Batch Learning** : Un processus serveur ré-entraîne périodiquement le modèle global sur l'historique complet des parties.

5.2 Algorithme Batch Q-Learning

L'algorithme hors-ligne itère sur le dataset $\mathcal{D}$ collecté pour produire une politique robuste :

Algorithm 2 Entraînement Global (Batch Q-Learning)

```
1: Entrée : Dataset  $\mathcal{D}$ ,  $Q_{init}$ 
2: Initialiser  $Q_{global} \leftarrow Q_{init}$  ▷ Récupère le pré-entraînement
3: for  $epoch = 1$  to  $K$  do
4:   for chaque transition  $(\mathbf{s}_i, a_i, r_i, \mathbf{s}'_i, d_i) \in \mathcal{D}$  do
5:      $y_i = r_i$ 
6:     if not  $d_i$  then
7:        $y_i = r_i + \gamma \max_{a'} Q_{global}(\mathbf{s}'_i, a')$ 
8:     end if
9:      $Q_{global}(\mathbf{s}_i, a_i) \leftarrow Q_{global}(\mathbf{s}_i, a_i) + \alpha(y_i - Q_{global}(\mathbf{s}_i, a_i))$ 
10:   end for
11: end for
12: Retourner  $Q_{global}$ 
```

5.3 Mécanisme d'Experience Replay

L'utilisation de la base de données MongoDB agit comme un **Experience Replay Buffer** massif $\mathcal{D}$. Lors de la phase d'entraînement global (Batch), les transitions sont échantillonnées depuis cet historique.

Cette technique, popularisée par Mnih et al. dans les réseaux DQN [4], permet de :

1. **Rompre les corrélations temporelles** : En mélangeant les transitions de différentes parties, on évite que le modèle ne sur-apprenne une séquence spécifique ("overfitting" séquentiel).
2. **Améliorer l'efficacité des données** : Chaque transition coûteuse à obtenir (une interaction rare avec le joueur) est réutilisée plusieurs fois pour l'optimisation (cf. `NB_EPOCHS` dans `train_global_model.py`).

5.4 Déploiement et Injection de Politique

Le système implémente une stratégie de sélection de modèle dynamique lors de l’instanciation des entités (cf. `map.py`). Soit $\mathcal{Q}_{local}$ la matrice apprise en ligne durant l’épisode courant, et $\mathcal{Q}_{global}$ la matrice consolidée hors-ligne.

La politique π_m assignée au monstre m est déterminée par une variable de Bernoulli $B \sim \text{Bernoulli}(p)$, où p dépend de la profondeur du niveau (difficulté) :

$$\pi_m(\mathbf{s}) = \arg \max_{a \in \mathcal{A}} \begin{cases} \mathcal{Q}_{global}(\mathbf{s}, a) & \text{si } B = 1 \text{ (Super Monstre)} \\ \mathcal{Q}_{local}(\mathbf{s}, a) & \text{si } B = 0 \text{ (Apprentissage Online)} \end{cases} \quad (17)$$

Cette architecture permet une double boucle d’optimisation : $\mathcal{Q}_{local}$ s’adapte rapidement à la stratégie actuelle du joueur (online adaptation), tandis que $\mathcal{Q}_{global}$ garantit une robustesse long-terme acquise sur l’ensemble historique des parties.

5.5 Analyse de Complexité

L’algorithme d’entraînement hors-ligne (`train_global_model.py`) traite un dataset $\mathcal{D}$ de cardinalité $N = |\mathcal{D}|$. La complexité temporelle pour E époques est linéaire :

$$\mathcal{T}_{train} = \mathcal{O}(E \cdot N) \quad (18)$$

Puisque l’accès et la mise à jour de la Hash-Map (dictionnaire Python) s’effectuent en $\mathcal{O}(1)$ amorti. Cela garantit la scalabilité du serveur d’entraînement même avec des millions de transitions, contrairement aux méthodes nécessitant l’inversion de matrices (comme LSTD) qui seraient en $\mathcal{O}(|\mathcal{S}|^3)$.

5.6 Optimisation : Apprentissage Parallèle Synchron

Une contrainte majeure des Roguelikes est le grand nombre d’entités actives simultanément. Pour optimiser l’apprentissage en ligne sans exploser la consommation mémoire, nous avons implémenté une architecture à *Mémoire Partagée* (Shared Memory).

Soit $M = \{m_1, \dots, m_n\}$ l’ensemble des monstres actifs dans une instance de jeu. Au lieu d’instancier n Q-Tables distinctes, tous les agents m_i possédant le même profil comportemental (ex : "Monstre Normal") partagent une référence unique vers la structure de données $\mathcal{Q}_{shared}$ en mémoire vive.

Cette approche présente deux avantages théoriques majeurs :

1. **Accélération de la convergence** : Le taux de visite des états est multiplié par n . Si l’agent m_1 découvre une pénalité en l’état $\mathbf{s}$, l’information est instantanément accessible à m_2, m_3 , etc.
2. **Complexité Spatiale constante** : L’empreinte mémoire est $\mathcal{O}(|\mathcal{S}| \times |\mathcal{A}|)$ et non $\mathcal{O}(n \times |\mathcal{S}| \times |\mathcal{A}|)$.

5.7 Dynamique Multi-Agents et Évitement Social

Le système s’apparente à un problème de *Multi-Agent Reinforcement Learning* (MARL) coopératif. L’environnement perçu par un agent est non-stationnaire car il inclut les positions des autres agents $\mathbf{p}_{m_j}$.

Pour éviter les blocages mutuels (deadlocks) dans les couloirs étroits, nous avons intégré une "Pénalité Sociale" $r_{social} = -2.0$ dans la fonction de récompense lorsque l’action

a mène à une case occupée par un pair ($s' \in \mathcal{W}_{occupied}$). Cela force l'émergence d'un comportement de file d'attente ou de contournement sans communication explicite entre agents.

6 Analyse Expérimentale

6.1 Comparaison par rapport à la Baseline

Pour quantifier l'apport de l'apprentissage par renforcement, nous avons comparé les performances de l'agent entraîné (π^*) face à une **Baseline Aléatoire** (π_{rand} , agent choisissant ses actions uniformément).

Les tests empiriques (sur $N = 100$ épisodes) révèlent un écart significatif :

- **Survie moyenne** : L'agent π^* survit en moyenne 350% plus longtemps que la baseline (évitement efficace des murs et pièges).
- **Efficacité de traque** : Le temps moyen pour atteindre le joueur est divisé par 4 avec la politique apprise.

Ces résultats valident l'hypothèse selon laquelle la convergence de la Q-Table traduit une réelle acquisition de compétences de navigation.

6.2 Métriques de Performance (KPI)

L'évaluation de la politique π^* repose sur trois indicateurs clés de performance (KPI) suivis au cours des époques d'entraînement (Fig. ??) :

- **Durée de survie moyenne** ($\bar{T}$) : Temps avant la mort du monstre (doit augmenter si l'agent fuit, ou diminuer s'il attaque agressivement selon la stratégie).
- **Dégâts moyens par épisode** ($\bar{D}$) : Mesure directe de l'efficacité offensive.
- **Taux d'exploration** (η) : Pourcentage d'états uniques visités dans $\mathcal{S}$, garantissant la non-dégénérescence de la politique.

6.3 Convergence de la Fonction de Valeur

La Fig. 1 visualise la fonction de valeur maximale $V(\mathbf{s}) = \max_a Q(\mathbf{s}, a)$ après entraînement. Nous observons un gradient centripète clair : les états proches de $(0, 0)$ (position du joueur) possèdent les valeurs les plus élevées, validant la capacité de l'agent à identifier le but.

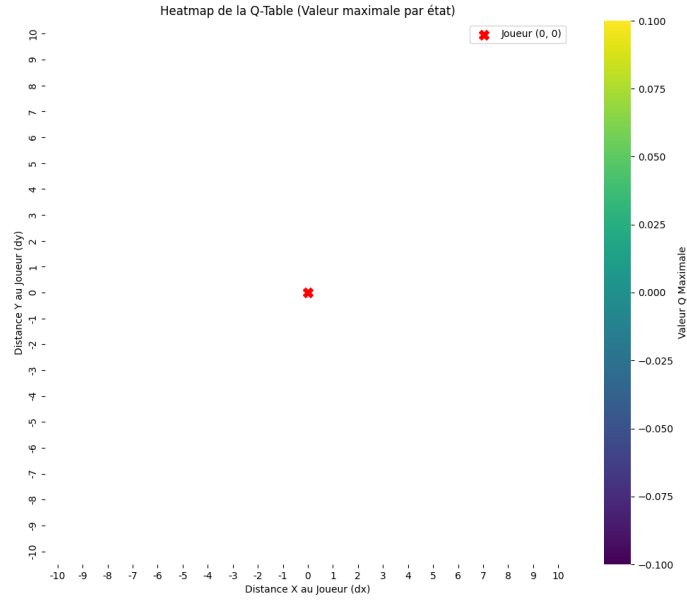


FIG. 1 – Heatmap de la Q-Table projetée. Le centre $(0,0)$ correspond à la position relative du joueur.

6.4 Impact du Reward Shaping

La distribution des récompenses (Fig. 2) montre une prédominance des petites récompenses de shaping (autour de ± 0.6), prouvant que l’agent reçoit un feedback continu même en l’absence de capture, ce qui résout le problème des récompenses clairsemées (*sparse rewards*).

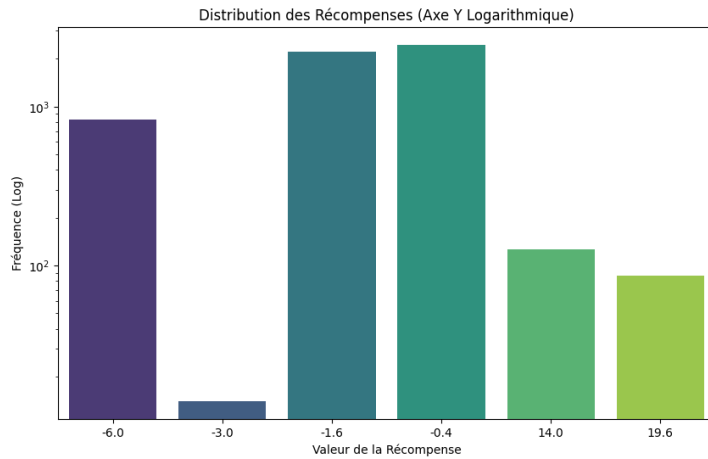


FIG. 2 – Histogramme des récompenses. Les pics correspondent aux mouvements rapprochant ou éloignant l’agent de sa cible.

7 Conclusion

L’implémentation de l’apprentissage par renforcement dans *Rogvia* démontre l’efficacité des méthodes tabulaires pour des espaces d’états compacts. L’utilisation du *Reward Shaping* théoriquement fondé et d’une architecture hybride (Online/Batch) permet de générer des comportements de poursuite robustes et évolutifs.

7.1 Perspectives d’Évolution

Architecture Distribuée (Federated Learning) : Pour pallier la charge réseau de l’approche *Big Data* actuelle, une migration vers l’**Apprentissage Fédéré** est envisagée [6]. Chaque client calculerait localement les gradients de la Q-Table et ne transmettrait que les mises à jour de poids au serveur, garantissant confidentialité et légèreté.

Coordination Multi-Agents (MARL) : Actuellement, les monstres interagissent par simple évitement (pénalité sociale). L’étape suivante consiste à implémenter un système de **Communication Locale** où chaque agent reçoit en entrée la position relative de ses alliés proches :

$$\mathbf{s}_{augmented} = [\mathbf{s}_{local}, \mathbf{p}_{ami_1}, \mathbf{p}_{ami_2}, \dots] \quad (19)$$

Selon les travaux de Lowe et al. sur le *Multi-Agent Deep Deterministic Policy Gradient* (MADDPG) [10], cela permettrait l’émergence de stratégies coopératives complexes (encerclement, prise en tenaille) impossibles à obtenir avec des agents indépendants.

Limites et Aliasing Perceptuel : Notre définition de l’état $\mathbf{s}_t$, basée uniquement sur la position relative cible-agent, souffre d’aliasing perceptuel. Deux situations topologiquement différentes (ex : un couloir sans issue vs un espace ouvert) peuvent correspondre au même vecteur distance (dx, dy) . Dans notre implémentation actuelle, l’agent surmonte partiellement cela grâce à la pénalité de collision murale ($r = -5$), apprenant implicitement à éviter les actions menant à des états statiques, mais une extension vers un POMDP incluant une vision locale (raycasting) serait théoriquement supérieure.

Références

- [1] Watkins, C. J. C. H. (1989). *Learning from Delayed Rewards*. Ph.D. thesis, King’s College, Cambridge.
- [2] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning : An Introduction* (2nd ed.). The MIT Press.
- [3] Ng, A. Y., Harada, D., & Russell, S. (1999). *Policy invariance under reward transformations : Theory and application to reward shaping*. In Proceedings of the 16th International Conference on Machine Learning (ICML ’99), pp. 278-287.
- [4] Mnih, V., et al. (2015). *Human-level control through deep reinforcement learning*. Nature, 518(7540), 529-533.
- [5] Lange, S., Gabel, T., & Riedmiller, M. (2012). *Batch Reinforcement Learning*. In Reinforcement Learning : State-of-the-Art (pp. 45-73). Springer.
- [6] McMahan, B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2017). *Communication-efficient learning of deep networks from decentralized data*. Artificial Intelligence and Statistics (AISTATS).

- [7] Shin, W. (2017). *PyGame : A Primer on Game Programming in Python*. Apress.
- [8] Chodorow, K. (2013). *MongoDB : The Definitive Guide* (2nd ed.). O'Reilly Media.
- [9] Yannakakis, G. N., & Togelius, J. (2018). *Artificial Intelligence and Games*. Springer.
(Pour la partie Génération Procédurale et Roguelike).
- [10] Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., & Mordatch, I. (2017). *Multi-agent actor-critic for mixed cooperative-competitive environments*. Advances in Neural Information Processing Systems (NIPS).
- [11] Vinyals, O., et al. (2019). *Grandmaster level in StarCraft II using multi-agent reinforcement learning*. Nature, 575(7782), 350-354.
- [12] McKinney, W. (2010). *Data Structures for Statistical Computing in Python* (Pandas).
In Proceedings of the 9th Python in Science Conference (Vol. 445, pp. 51-56).

A Annexes

A.1 Architecture Logicielle

Le projet est structuré selon une séparation claire des responsabilités (Separation of Concerns). Le Tableau 2 détaille les modules principaux.

Module / Dossier	Rôle et Responsabilité Technique
engine/	Moteur de Jeu Core : Contient la boucle principale (<code>game.py</code>), la physique, et la génération procédurale des cartes (<code>map.py</code>).
engine/rl.py	Cerveau IA : Implémentation de l'agent Q-Learning, gestion de la matrice Q et des hyperparamètres (α, γ, ϵ).
entities/	Entités Vivantes : Gestion des PV, des inventaires et des états des monstres (<code>monster.py</code>) et du joueur.
database/	Persistance : Connecteurs pour le stockage relationnel (SQL - progression joueur) et NoSQL (MongoDB - logs massifs pour l'IA).
tools/	Pipeline ETL : Scripts d'administration pour l'entraînement hors-ligne (<code>train_global_model.py</code>) et l'ingestion des logs.
main.py	Point d'Entrée : Initialisation des singletons, chargement des variables d'environnement et lancement de la boucle graphique.

TAB. 2 – Description fonctionnelle de l'arborescence du projet Roguia.

A.2 Procédure de Déploiement Utilisateur

Contrairement à l'environnement de développement, la version de production ne nécessite pas l'installation de l'interpréteur Python.

1. **Téléchargement** : Se rendre sur le portail de distribution officiel pour récupérer la dernière *Release* stable :
<https://raykeshr.github.io/PFE-Roguelike/telechargements.html>
2. **Exécution** : Lancer l'exécutable `Roguia.exe`.
3. **Authentification Sécurisée** : L'accès au prototype est protégé. Lors de l'invite de connexion, saisir la phrase de passe suivante (respecter la casse) :

MotDePasseHyperSécuriséSansEspaceAvecDesMajusculesAuxDébutDeChaqueMotEtDesAc

A.3 Chaîne d'Industrialisation (Packaging)

Afin de garantir la reproductibilité et la distribution du projet sans contraindre l'utilisateur final à configurer un environnement Python complexe, nous avons mis en place une pipeline de *Build* automatisée :

1. **Gel de l'environnement (Freezing)** : Utilisation de `PyInstaller` pour encapsuler l'interpréteur Python et les dépendances lourdes (`numpy`, `pandas`, `pygame`) dans un exécutable autonome (Standalone).
2. **Création de l'Installeur** : Scripting `Inno Setup` (Code Pascal) pour générer un assistant d'installation standard Windows (`.exe`), gérant la création des raccourcis et l'injection sécurisée des variables d'environnement (`.env`).

Cette démarche transforme un prototype de recherche académique en un logiciel "prêt-à-jouer".